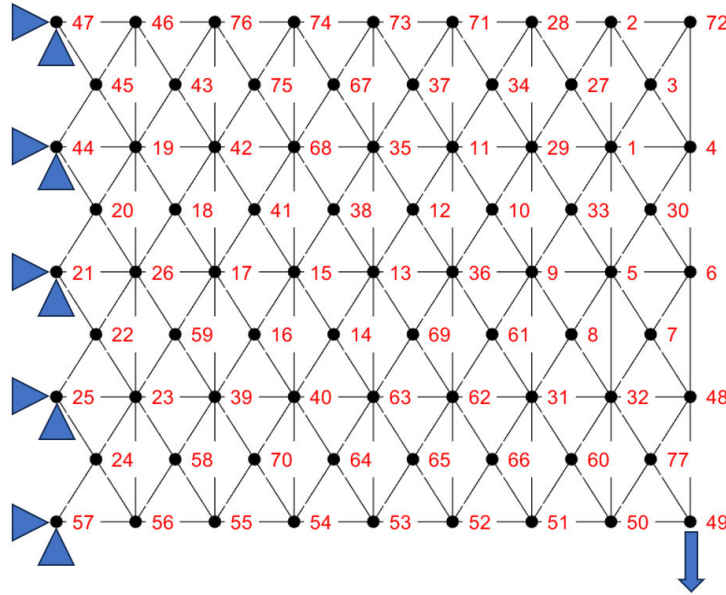


```
close all; clear; clc
```

Structural optimization of a truss structure

You are asked to **size the cross sectional areas** of a truss structure in order to **minimize its compliance** under **volume constraint**, i.e. $V - V_0 \leq 0$. The truss resembles a **cantilever beam** and is made by **77 nodes** (of which you see the labels) and **200 elements**. A vertical load $F_y = -6'000$ N is acting on **node 49** and **nodes 47, 44, 21, 25 and 57** are hinged to the ground (their two degrees of freedom are suppressed).



Requests

- First request

Optimize the truss for the **four cases presented below**. For each case **draw the optimized structure**, plot the value of the **compliance C** and **volume fraction α** at each iteration. When drawing the structure, it is suggested to use a line width proportional to the cross sectional area.

Case	α	lb [mm ²]	ub [mm ²]
1	0.05	0.0001	800
2	0.10	0.0001	800
3	0.15	0.0001	800
4	0.20	0.0001	800

ub and **lb** are the **upper and lower bounds** the for the cross-sectional areas.

$$lb \leq x_j \leq ub$$

α is the volume fraction of the structure, defined as $V_0 = \alpha V_{\max}$, with $V_{\max}$ = maximum value the truss can achieve $= \sum_{j=1}^{200} l_j = 800 \sum_{j=1}^{200} l_j$ where l_j is the **length** of the j -th bar element.

The **compliance** of the truss is the elastic strain energy, given by:

$$C = F^T u.$$

Where F is the vector of external loads and u the vector of nodal displacements.

- **Second request**

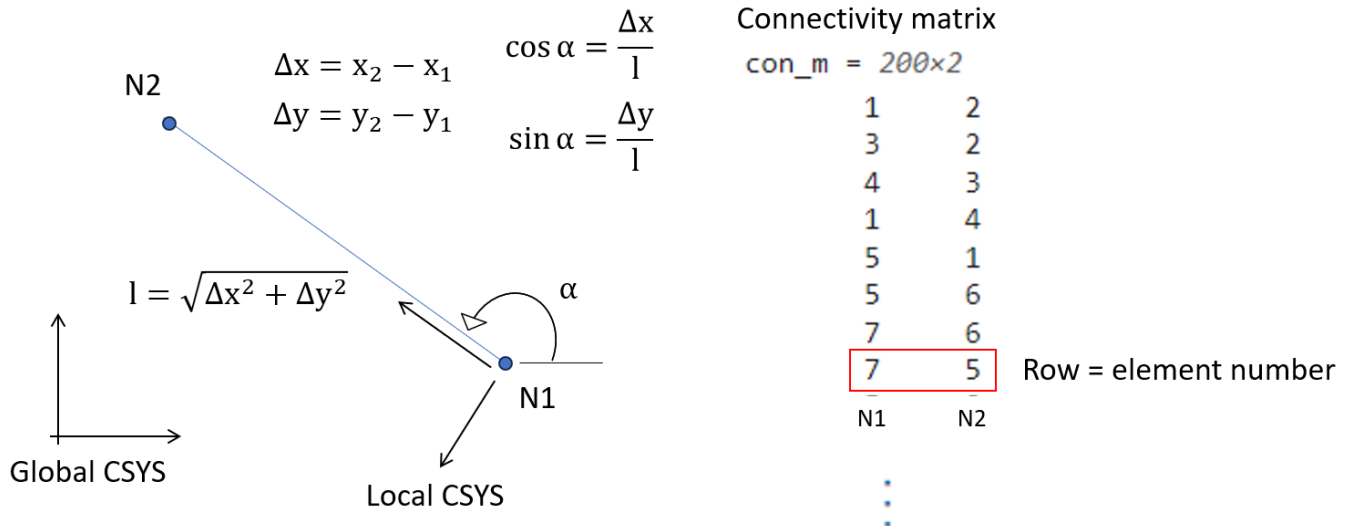
Report the **values** of the **cross sectional areas** and **stresses** acting in **elements 64, 77, 82**. What do you notice?

Pre-processing

On WeBeep you find two files:

- **NodeCoord_Cantilever_mm.txt**, which contains the list of the nodal coordinates expressed in millimeters;
- **ElementNodes_Cantilever.txt**, the connectivity file.

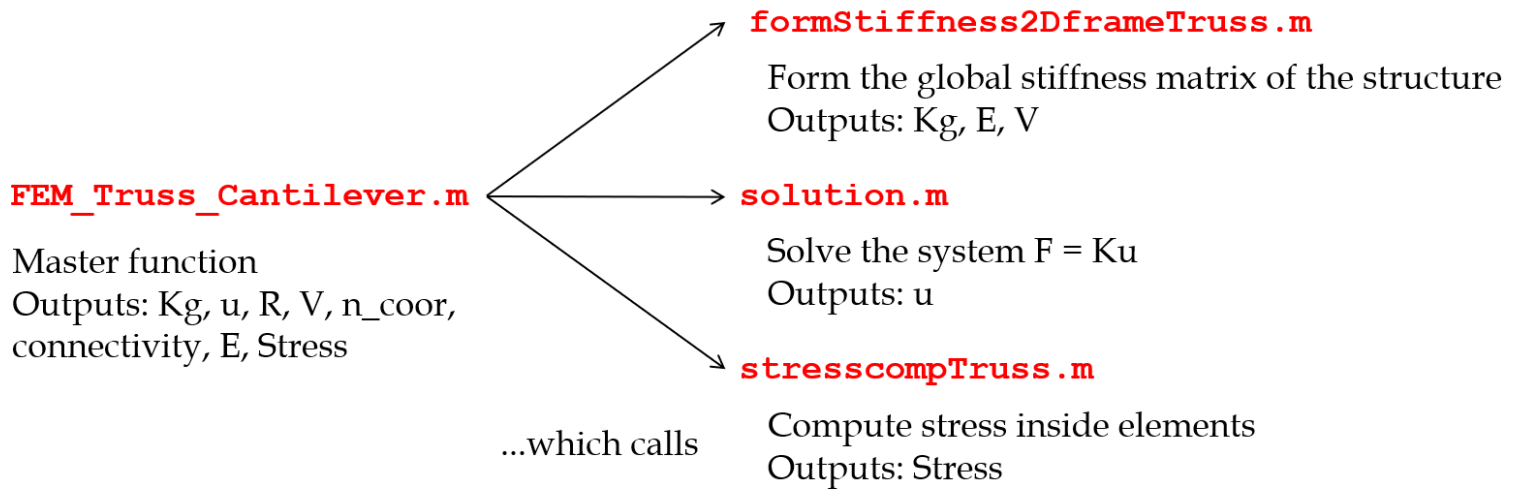
Each row j of the connectivity file contains the labels of nodes forming the j -th element.



1. For element j , identify node 1 and 2 looking at row j of the **connectivity matrix**;
2. Extract the **coordinates of node 1 and 2 from the table of nodal coordinates**. Enter in the nodal coordinate table at rows equal to nodal labels (in the example, rows 7 and 5);
3. Compute Δx and Δy **respecting** the subtraction order (node 2 – node 1), calculate the length of the element and the sinus and cosinus associated to the inclination angle.

Moreover, Matlab functions that run the finite element analysis of the truss are uploaded:

- FEM_Truss_Cantilever.m (master)
- formStiffness2DframeTruss.m (slave)
- solution.m (slave)
- stresscompTruss.m (slave)



Degrees of freedom

Any truss structure - made by bars linked together by hinges - has $2N_{\text{nodes}}$ degrees of freedom. In the Global CSYS, each node is described by two displacements (one along X and one along Y).

The formulae which links a node to its corresponding global degrees of freedom are:

$$\begin{bmatrix} \text{dof 1} \\ \text{dof 2} \end{bmatrix}_{\text{node } n} = \begin{bmatrix} u_n \\ v_n \end{bmatrix} = \begin{bmatrix} 2n - 1 \\ 2n \end{bmatrix}$$

Usage of the provided Matlab functions

As first, you should perform a finite element analysis of your structure. The function which allows you to do that is called `FEM_Truss_Cantilever.m`.

Inputs

The inputs are:

```

% x vector of cross-sectional areas
% F vector of external forces
%
x = 100*ones(200, 1); % number of elements
F = zeros(154, 1); % number of degrees of freedom
%
% external force on node 49, direction 2
P = -6000;
F(49*2) = P;
  
```

Run the FEA:

```
[Kg, u, R, coor_n, V, connectivity, E, sigma] = FEM_Truss_Cantilever(x,F);
```

With the Matlab function `spy(Kg)` you can have a very fast check on the computed matrix `Kg` (only non null elements are displayed).

```
spy(Kg)
```

Outputs:

```
% Kg global stiffness matrix [n_dof x n_dof]
% u vector of displacements [n_dof]
% R vector of reaction forces [constrained dof]
% coor_n matrix of nodal coordinates [n_nodes x 2]
% V volume of the truss [scalar]
% connectivity matrix [n_elements x 2]
% E elastic modulus [scalar]
% sigma column vector [n_elements]
```

Solution procedure

1. Initialize data; draw the structure; compute values that are constant;
2. Initialize a cycle for N iterations;
3. Run a F.E.A. (use the provided Matlab function);
4. Compute the **sensitivity** of the compliance;
5. Solve the dual problem;
6. Iterate again the cycle (point 3.);
7. Check if the solution has converged (are the variations of compliance almost null in the last iterations?);
8. Draw the structure obtained for the last iteration.

The for loop should have this structure. We will see it in details.

```
for jj = 1 : N_iter
    % Run the F.E.A.
    ...
    % Sensitivity analysis
    ...
    % Dual problem solution
    ...
    % Saving the solution
    ...
end
```

Run the F.E.A.

Run the finite element analysis of the truss and **compute the compliance**. You can use the function FEM_Truss_Cantilever.m provided.

```
Ce = F'*u; % compliance of the structure
```

Sensitivity analysis

To linearize the problem, you have to compute the sensitivity of the compliance.

$$\frac{\partial C}{\partial x_j} = \frac{\partial}{\partial x_j} \mathbf{F}^T \mathbf{u} = \mathbf{u}^T \mathbf{K} \frac{\partial \mathbf{u}}{\partial x_j} = \dots = -\mathbf{u}_j^T \mathbf{k}_j^0 \mathbf{u}_j.$$

Where $\mathbf{u}_j$ is the vector of nodal displacements (in the global reference frame) of the j -th element. While $\mathbf{k}_j^0$ is defined as:

$$\mathbf{k}_j^0 = \frac{E}{l_j} \begin{bmatrix} \cos^2(\theta_j) & \sin(\theta_j)\cos(\theta_j) & -\cos^2(\theta_j) & -\sin(\theta_j)\cos(\theta_j) \\ \sin(\theta_j)\cos(\theta_j) & \sin^2(\theta_j) & -\sin(\theta_j)\cos(\theta_j) & -\sin^2(\theta_j) \\ -\cos^2(\theta_j) & -\sin(\theta_j)\cos(\theta_j) & \cos^2(\theta_j) & \sin(\theta_j)\cos(\theta_j) \\ -\sin(\theta_j)\cos(\theta_j) & -\sin^2(\theta_j) & \sin(\theta_j)\cos(\theta_j) & \sin^2(\theta_j) \end{bmatrix}$$

Where θ_j is the element's inclination angle.

```
% Sensitivity analysis
dCe = zeros(n_elem, 1);
for ii = 1 : n_elem
    n1 = connectivity(..); n2 = ..;
    x1 = coor_n(..); y1 = coor_n(..);
    x2 = coor_n(..); y2 = coor_n(..);
    deltaX = ...
    deltaY = ...
    cosa = ... % cosine
    sena = ... % sine
    l = ... % length
    k0 = E/l*[... % stiffness matrix (global reference frame)
    dof_e = [... % global degrees of freedom of element ii
    ue = u(... % nodal displacements of the element ii, global CSYS
    dCe(ii) = - ue' * k0 * ue; % sensitivity
end
```

Solve the dual problem

In order to solve the dual problem you should **build the derivative of the dual function** `dphi.m`. You can either define this function at the end of your script or create another file.

From the CONLIN approximation results:

$$x_j = \sqrt{\frac{q_{0j}}{\lambda l_j}}$$

Where $q_{0j} = -x_j^2 dCe_j = x_j^2 \mathbf{u}_j^T \mathbf{k}_j^0 \mathbf{u}_j$.

```
function [y, x_new] = dphi(lambda, x, lb, ub, dCe, l, v0)
    if lambda < 0
        lambda = 0;
    end
    q0 = -x.^2 .* dCe;
    x_trial = sqrt(...
    % check on upper bound
    x_trial(x_trial > ub) = ub;
    % check on lower bound
    x_trial(x_trial < lb) = lb;
    % x_new
```

```

x_new = x_trial;
% constraint (equal to dphi/dlambda)
y = l'*x_new - V0; % constraint on volume
end

```

Once you have built the function dphi you can solve the problem.

```

l_sol = fsolve(@(lambda) dphi(lambda, x, lb*ones(n_elem, 1), ub*ones(n_elem, 1),
dCe, l, V0), l0);
[~, x_sol] = dphi(l_sol, x, lb, ub, dCe, l, V0);

```


Final overview

The overall code should look like this:

```
close all; clear; clc
% data
n_elem = 200;
n_nodes = 77;
x = 100*ones(n_elem, 1);
F = zeros(2*n_nodes, 1);
P = -6000;
F(49*2) = P;
[Kg, u, R, coor_n, V, connectivity, E, sigma] = FEM_Truss_Cantilever(x, F);
l_mean = V/(n_elem*100); % average length
%
alpha = 0.05; % volume fraction
%
N = 100; % number of iterations
l0 = 100; % Lagrangian multiplier initialization
ub = 800; lb = 0.0001; % lower and upper bounds for the D.V.
Vmax = n_elem*l_mean*ub; % maximum volume
V0 = Vmax*alpha; % constraint, V - V0 <= 0
A0 = V0/(n_elem*l_mean); % cross-sectional area initialization WHY IS A0 NEEDED?
x = A0*ones(n_elem, 1); % column vector of the areas
%
%
%
for jj = 1 : N
    [Kg, u, R, coor_n, V, connectivity, E, sigma] = FEM_Truss_Cantilever(x, F);
    Ce(jj) = ...
    % Sensitivity Analysis
    dCde=zeros(n_elem, 1);
    for ii = 1 : n_elem
        n1 = connectivity(..); n2 = ..;
        x1 = coor_n(..); y1 = coor_n(..);
        x2 = coor_n(..); y2 = coor_n(..);
        deltaX = ...
        deltax = ...
        cosa = ... % cosine
        sena = ... % sine
        l = ... % length
        k0 = E/l*... % stiffness matrix (global reference frame)
        dof_e = [... % degrees of freedom of element ii
        ue = u(... % nodal displacements of the element ii, global CSYS
        dCe(ii) = - ue' * k0 * ue; % sensitivity
    end
    %
    % Solve the dual problem
    l_sol = fsolve(@(lambda) dphi(lambda, x, lb, ub, dCe, l, V0), l0);
    [~, x_sol] = dphi(l_sol, x, lb, ub, dCe, l, V0);
```

```

%
% Save the variables
volume(jj) = V;
alfa(jj) = V/Vmax;
area(:, jj) = x_sol;
stress(:, jj) = sigma;
x = x_sol;
l0 = l_sol;
end
% plot the requests (volume, compliance, stress, areas...)
figure()
...
%
function y= dphi(...)
    ...
end

```